AI Lab - Uniformed Search Strategies

Alessandro Farinelli
Thanks to Davide Corsi, Luca Marzari and Celeste Veronese for help with slides and code

University of Verona
Department of Computer Science



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

Assignments

- Your assignments for this lesson can be found at: *uninf-search/uninf_search_1_problem.ipynb*. You will be required to implement some Uninformed Search algorithms
- In the following you can find pseudocodes for such algorithms

Uninformed Search: tree and graph search versions

function TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier

---

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

Node data structure

Search algorithms require a data structure to keep track of the search tree. A *Node* in the tree is represented by a data structure with three components:

Node(state, parent, pathcost)

- state: the state to which the node corresponds;
- parent: the node in the tree that generated this node;
- pathcost: the total cost of the path from the initial state to this node
- depth: the depth of the node in the search tree. You do not need to initialize this, as this is automatically set by the constructor.

Breadth-First Search (BFS): graph search version

Require: $problem$
Ensure: $solution$

1: $node \leftarrow$ a node with STATE $= problem.$INITIAL-STATE, PATH-COST $= 0$
2: **if** $problem.$GOAL-TEST$(node.$STATE$)$ **then return** SOLUTION$(node)$
3: $frontier \leftarrow$ NODE-QUEUE
4: $explored \leftarrow \emptyset$
5: **while** not IS-EMPTY$(frontier)$ **do**
6: $\quad node \leftarrow$ REMOVE$(frontier)$ ▷ Remove last node
7: $\quad explored \leftarrow explored \cup node.STATE$
8: $\quad$ **for each** $action$ **in** $problem.$ACTIONS$(node.$STATE$)$ **do**
9: $\quad\quad child \leftarrow$ CHILD-NODE$(problem, node, action)$
10: $\quad\quad$ **if** $child.$STATE **not in (** $explored$ **or** $frontier$ **)** **then**
11: $\quad\quad\quad$ **if** $problem.$GOAL-TEST$(child.$STATE$)$ **then return** SOLUTION$(child)$
12: $\quad\quad\quad frontier \leftarrow$ INSERT$(child)$
return FAILURE

Iterative Deepening Search (IDS): tree search version

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```